Project Documentation: SecureDoc-RAG

1. Introduction

SecureDoc-RAG is an enterprise-grade document query system built with Retrieval-Augmented Generation (RAG). It enables multi-tenant document management where users can upload confidential documents and receive precise answers powered by Google Gemini 2.5 Flash and Pinecone vector database. The system is designed with strict data isolation, automated Personal Identifiable Information (PII) masking, and robust anti-prompt-injection safeguards.

2. Core Features & Capabilities

• **Multi-Tenant Architecture**: Supports multiple users with complete data isolation. Cross-tenant data leakage is prevented at the vector database level.

• **Automated PII Masking**: Sensitive information such as emails, passwords, phone numbers, credit cards, and IDs (PAN, Aadhaar) are automatically redacted before processing or embedding.

• **RAG-Powered AI Q&A**: Uses state-of-the-art LLMs to answer natural language questions based directly on the uploaded documents.

• **Multi-Chat Session Management**: Create, rename, switch between, and delete multiple conversation threads with persistent history and message linkage.

• **Interactive Source Citations**: AI responses cite exact retrieved document chunks with similarity scores. Clickable badges open a dedicated side panel with chunk excerpts and full sanitized document viewer.

• **Downloadable Q&A Report Export**: Export conversation transcripts and Q&A summaries as formatted PDF (via ReportLab) or Microsoft Word DOCX (via python-docx) reports.

• **Document Versioning & History**: Automatic version incrementing (**v1**, **v2**, etc.) on re-uploading documents with the same name, archiving previous iterations, replacing old vector embeddings in Pinecone/pgvector, and tracking version history.

• **Anti-Prompt Injection Defense**: Untrusted document contents are securely wrapped in system boundaries to prevent malicious instructions from hijacking the AI model.

3. System Architecture

The system architecture follows a secure and isolated pipeline:

1. Document Upload & Versioning: Users upload PDF or DOCX files. If an existing filename is detected, the version increments and old vectors are archived/replaced in Pinecone & DB.

2. Text Extraction & Masking: Text is extracted using PyMuPDF and python-docx. The text is immediately passed through a PII Masking Engine.

3. Chunking & Embedding: Cleaned text is chunked using LangChain and converted into vector embeddings via Gemini.

4. Vector Storage: Embeddings are stored in Pinecone and pgvector/SQLite with strict metadata tagging (**user_id**, **document_id**, **chunk_index**).

5. Session-Linked Query & Retrieval: User questions are embedded and vector search is performed with mandatory **user_id** tenant isolation filter.

6. Generation & Citations: Retrieved context chunks and question are sent to Gemini to generate accurate answers with verifiable chunk citations.

7. Report Export: Conversations can be exported into branded PDF or DOCX reports with metadata, metrics, and citation references.

4. Security & Data Isolation

The primary focus of SecureDoc-RAG is data security.

• **Mandatory Pinecone Tenant Isolation**: Every chunk indexed stores strict metadata. Vector retrieval always applies a mandatory metadata filter (**{"user_id": {"$eq": "user_id"}}**), making cross-tenant search impossible.

• **Regex PII Masking Engine**: Data is sanitized on the fly. Example transformations:

- Emails ➔ **[EMAIL]**
- Credit Cards ➔ **[CARD]**
- Passwords ➔ **[PASSWORD]**
- Aadhaar Cards ➔ **[AADHAAR]**

5. Technology Stack

Frontend

- **Framework**: React 19 + Vite
- **Styling**: Tailwind CSS v4 & Vanilla CSS Design System
- **State Management**: TanStack Query (React Query) v5

Backend

- **Framework**: FastAPI (Python 3.12+)
- **Database**: SQLAlchemy 2.0 (SQLite / PostgreSQL pgvector)
- **Document & Report Generation**: ReportLab (PDF) & python-docx (DOCX)
- **Authentication**: JWT & Passlib bcrypt

AI & Data Pipeline

- **LLM / Embeddings**: Google Gemini 2.5 Flash / **text-embedding-004**
- **Vector DB**: Pinecone Serverless & pgvector
- **Chunking**: LangChain **RecursiveCharacterTextSplitter**

6. Setup & Deployment

The system requires Node.js 18+ and Python 3.10+.

- **Environment**: Keys required include **GEMINI_API_KEY**, **PINECONE_API_KEY**, and **JWT_SECRET**.
- **Backend Start**: **uv run uvicorn backend.app.main:app --reload --port 8000**
- **Frontend Start**: **cd frontend && npm install && npm run dev**

7. API Reference Summary

- **POST /api/auth/register** - Register a new tenant.
- **POST /api/auth/login** - Obtain JWT token.
- **POST /api/chat/sessions** - Create a new chat conversation thread.
- **GET /api/chat/sessions** - List user chat conversation threads.
- **PATCH /api/chat/sessions/{id}** - Rename chat conversation title.
- **DELETE /api/chat/sessions/{id}** - Delete a chat session and its messages.
- **POST /api/chat** - Query RAG system with session linking and citation returns.
- **GET /api/chat/export** - Export conversation transcript as downloadable PDF or DOCX.
- **POST /api/documents/upload** - Securely upload and process documents with automatic versioning.
- **GET /api/documents** - List documents (**include_archived** filter support).
- **GET /api/documents/{id}/history** - Get complete document version history.
- **GET /api/documents/{id}/view** - View PII-masked document content.
- **DELETE /api/documents/{id}** - Delete document and remove vector embeddings.

8. Testing & Validation

The project includes a robust Pytest suite testing PII masking logic, tenant vector isolation, and API endpoint security to ensure production-level reliability.